MCU_Mem_01

Topics:
-RAM -Virtual Memory/Paging -Segmentation -Microcontroller Context -NAND /NOR Flash

- **RAM:** (Fastest, Main memory, Volatile, Random access by individual bytes)

Dynamic RAM (DRAM): The memory cells use tiny electrical **capacitors** to store data.
These capacitors leak charge and must be **refreshed** dynamically thousands of times per second to keep the data alive.

DDR: Double Data Rate RAM.
This RAM called DDR, bcz the Data is transferred on both the Rising and Falling Edge of the Clock Cycle.

CAS (Column Address Strobe) Latency: It is the no. of clock cycles between
- The memory controller sending a **READ** command to **DDR** memory.
- The first piece of requested **data** appearing on the data **bus**.

DIMM: Dual In-line Memory Module

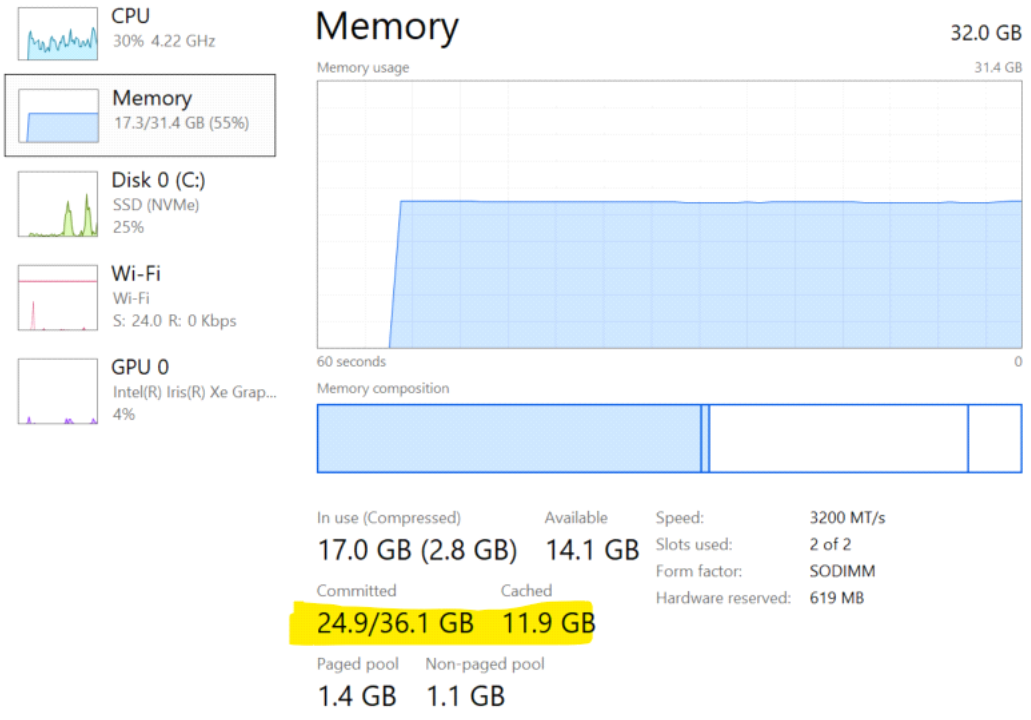
Static RAM (SRAM): This memory is much faster, but it is too expensive and bulky to be used for main system memory.
SRAM is instead used for your processor's internal **CPU Cache**.

SDRAM: Synchronous Dynamic Random-Access Memory (SDRAM), **evolved** DRAM.
All modern computer memory (including DDR4 and DDR5) is technically **SDRAM**.
[Traditional DRAM] ⟶ Asynchronous ⟶ CPU waits for RAM to respond
[Modern SDRAM] ⟶ **Synchronous** ⟶ RAM syncs with CPU **Clock** Grid

-When the System runs out of physical RAM, there is a technique called **Virtual Memory (swapping / Paging)** which is used to use a part of SSD (NVMe) space to behave as RAM.
-This prevents the program from Crashing.
-This is called **Page file** or **Swap Space**.
-In this **"Swapping"** process, the OS checks the unused or Idle Apps, copies their data and shuffles them out of the fast RAM onto the slower SSD Pagefile.

Feature	RAM (DDR5 / SDRAM)	Flash Memory (NAND / NOR)
Design	RAM stores data using a single transistor and a tiny **capacitor.** This electrical switch happens almost **instantly**	Flash memory stores data by trapping electrons inside an insulated cage called a **floating gate**. To change a "1" to a "0", it has to force electrons through an insulating barrier using high voltage. Pushing physical particles through a barrier takes significantly more time.
Speed	Fastest	Comparatively slower
Use	System Main Memory	SSD, NVM, Flash

Data Storage Mechanism	Charges a tiny **capacitor** (like a miniature battery).	Traps electrons inside an insulated **floating gate** / charge trap.
Write Process	**Direct Overwrite:** Changes any byte instantly.	**Erase-Before-Write:** Must wipe an entire block of data before rewriting.
Voltage Required	**Very Low (1.1V):** Minimal power needed to flip electrical charges.	**High Voltage:** Needs a strong burst to force electrons through insulation.
Data Lifespan (Wear)	**Infinite:** Can be read/written trillions of times without degrading.	**Finite:** Insulating layers wear out over time (limited write cycles).
CPU Connection	Direct connection via a massive, dedicated **Memory Bus**.	Indirect connection via a **Storage Controller** (NVMe/SATA protocols).
Average Latency	**~10 – 15 nanoseconds** (Near-instant response).	**~10 – 100 microseconds** (1,000× slower response than RAM).
Data Transfer Speed	**Up to 60,000+ MB/s** (DDR5).	**Up to 7,000 – 14,000 MB/s** (High-end Gen 4/5 SSDs).
Volatility	**Volatile:** Wipes completely when power is turned off.	**Non-Volatile:** Retains data permanently without power.
	leaky, open gates	locked, insulated cages.



```
Committed:  18.5 / 40.2 GB
             |       |
             |       └─ Total Limit (Physical RAM + SSD Pagefile combined)
             └─────────── Current Memory Demand (What apps are asking for)
```

Virtual Memory Concept:

The OS divides the Programs into Pages (4 KB size).
System maintains a master index called a **Page Table**.

1. Demand Paging (Loading on Demand)

When you launch an application, the operating system does not load the entire multi-gigabyte software into your RAM all at once. Instead, it uses **Demand Paging**—it only loads the specific pages of code the CPU needs *right this second* into physical RAM. The rest of the program remains asleep on the SSD until you click a feature that requires it.

2. Page Faults (The Missing Data Alert)

If the CPU executes a command and requests a page of data that isn't currently sitting in the physical RAM, a **Page Fault** occurs.

- This is not an error or a crash; it is simply an internal hardware signal.
- The CPU pauses the application for a brief millisecond while the operating system rushes over to the SSD, locates the missing page, and copies it into an open slot in the RAM. Once loaded, the CPU resumes work safely.

3. Page Replacement (Making Space)

If a Page Fault happens but your physical RAM is completely full, the operating system must decide what to kick out to make room. It uses a **Page Replacement Algorithm** (most commonly **LRU** - *Least Recently Used*).

- It identifies a page of data belonging to an idle background app (like a web browser tab you haven't looked at in an hour).
- It writes that idle page onto your SSD's swap space and completely clears it from the RAM.
- It then uses that newly emptied RAM slot to host the active data the CPU is currently begging for.
- Virtual memory gives Crash prevention, Isolation to every application its own private memory,

  And Cost, that can be saved over SSD use, by using the swapping technique.

✸ Segmentation:

segmentation is a memory management technique where a computer's memory is divided into variable-length, logically distinct chunks called **segments**.

1. The Core Concept: Logical Divisions

When you write code, your program isn't just one long line of data. It is naturally divided into different functional modules. Segmentation creates a specific memory container (a segment) for each of these logical pieces:

- **Code Segment:** Stores the actual compiled instructions of the application (read-only).
- **Data Segment:** Holds the global and static variables used by the program.
- **Stack Segment:** Handles temporary data like function parameters, local variables, and return addresses.
- **Heap Segment:** Dynamically allocates memory while the program is running.



2. How Segmentation Works (The Segment Table)

To keep track of these irregularly sized chunks, the CPU and Operating System use a hardware structure called a **Segment Table**. Every memory address generated by a program consists of a pair: **(Segment Number, Offset)**.

- **Segment Number (s):** Acts as the ID index for the table to find which chunk of memory to look in.
- **Offset (d):** The exact distance from the beginning of that specific segment to the target piece of data.

The Segment Table contains two vital guardrails for each segment:

- **Base Address:** The physical starting location of the segment in your laptop's actual RAM.
- **Limit (Size):** The exact length of the segment. If a program attempts to look at an offset larger than the limit (e.g., trying to access the 105th byte of a 100-byte segment), the CPU triggers a **Segmentation Fault (SegFault)** and instantly halts the app to prevent memory corruption.

3. Segmentation vs. Paging

While discussing **Virtual Memory** earlier, we looked at **Paging**. Both are ways to manage memory, but they approach the problem differently:

Feature	Segmentation	Paging
Size of Blocks	**Variable Length** (Depends on the size of the code module).	**Fixed Size** (Always exactly 4 KB blocks).
Who Decides It?	**The Programmer/Compiler** (Based on logical code structures).	**The Operating System/Hardware** (Invisible to the program).
The Main Problem	**External Fragmentation:** Leaves small, random gaps of empty space between segments that are too small to use.	**Internal Fragmentation:** Small files still take up a full 4 KB page, wasting tiny bits of space inside the page.

Microcontroller Context:

-No Virtual Memory: Microcontrollers execute code directly on physical hardware addresses. There is no Operating System or Memory Management Unit (MMU) translating logical addresses to physical RAM.

-**Segmentation** does not exist in standard microcontrollers. Instead MPU (Memory Protection Unit) is there.

-**Fixed Memory Regions (Harvard/Von Neumann Architecture):** Instead of dynamic segments managed by a table, microcontrollers use fixed physical memory mappings defined by the hardware layout. For example, Flash memory (Code/RODATA) starts at a specific address (e.g., 0x08000000 on STM32), and internal SRAM (Variables/Stack/Heap) starts at another fixed address (e.g., 0x20000000).

-**The Exception (MPU):** High-end microcontrollers include a **Memory Protection Unit (MPU)**. The MPU allows developers to define rigid, fixed zones of RAM as "read-only" or "privileged." If code violates these zones, it triggers a hardware exception (like a **HardFault**), which is the embedded equivalent of a Segmentation Fault.

-Because microcontrollers lack an MMU to throw a traditional OS "Segmentation Fault," memory violations cause **HardFaults**, **BusFaults**, or **UsageFaults**.

-The type of **Paging** used in laptops—where the operating system dynamically cuts up code into **4KB blocks** and shuffles them between **RAM** and an **SSD—is never used in Microcontrollers**.

-Microcontrollers like the Infineon AURIX TC4D7XP or STM32 **avoid virtual paging** due to strict real-time constraints: Determinism (to avoid delays) and No Secondary Storage (On-Chip Flash Execution).

-Instead of paging, modern chips use **Static Memory Mapping** combined with an **MPU (Memory Protection Unit)**.

```
[ 0x00000000 to 0x07FFFFFF ] ──► System Boot / Vector Tables
[ 0x08000000 to 0x1FFFFFFF ] ──► On-Chip Flash Memory (Your Code)
[ 0x20000000 to 0x3FFFFFFF ] ──► Internal SRAM (Your Variables)
[ 0x40000000 to 0x5FFFFFFF ] ──► Hardware Peripherals (CAN, SPI, Timers)
```

✓ **Segmentation** = A logical view of memory as separate regions (Code, Data, Stack, Heap).

```
Program
├── Code Segment
├── Data Segment
├── Heap Segment
└── Stack Segment
```

✓ **Paging/Page Tables** = A mechanism that maps **virtual** addresses to **physical** RAM addresses.

Virtual Memory

Programs do not usually access physical RAM directly.

Instead:

```
Application
    |
Virtual Address
    |
MMU
    |
Physical Address
    |
RAM
```

Example:

```
Application accesses:
0x1000

MMU translates to:
0xA4501000

RAM receives:
0xA4501000
```

✓ **RAM itself does not inherently contain pages or segments.**
They are **architectural concepts** implemented by the CPU + MMU + OS, but the page table data structure resides in physical RAM.

✓ There is No MMU in Microcontrollers, that's why there is **no Virtual memory & no Page Tables.**
Addresses are mostly Physical.

✓ **Page Table is stored in physical RAM.** Each Page size is **4 KB**.

```
RAM
+--------------------+
| Program Data       |
+--------------------+
| Kernel             |
+--------------------+
| Page Table         |
+--------------------+
| Buffers            |
+--------------------+
```

CPU register:

```
CR3 (x86)

contains
Page Table Base Address
```

```
Page Table

VPN (Virtual Page)    PFN (Physical Frame)
-----------------------------------------
0                     15
1                     97
2                     44
3                     201
```

Ex: When CPU accesses virtual addresss: 0x1234,
VPN = 1, Offset = 0x234
VPN 1 --> PFN 97
Physical Address = 97* Page_size + offset = [97 * 4KB + 0x234]

✸ NOR vs NAND Flash:

It says about how the Flash cells are connected internally.

Feature	NOR Flash	NAND Flash
Access Type	Random Access	Block/Page Access
Read Speed	Fast random read	Fast sequential read
Write Speed	Slower	Faster
Erase Speed	Slower	Faster
Density	Lower	Higher
Cost per Bit	Higher	Lower
XIP (Execute In Place)	Supported	Not supported
Capacity	Typically smaller	Typically larger
Reliability	Better read reliability	Better storage density
Typical Size	KB to hundreds of MB	GB to TB
Usage	In Microcontrollers' BootLoader, Application Firmware, Configuration, Calibration Data	Linux RootFS, Android Image, Video Files, Logs, User Data

Why in Embedded to store Program/Code the NOR flash is used because it can be accessed Byte wise. But NAND flash can be used only Page or Block wise.

-----X--------- E N D ------X-------